

Database Trending Scenario Interview Guide

22 practical SQL questions with solutions • Oracle & PostgreSQL • Senior Java / Spring Boot

Answer framework: Clarify assumptions → write SQL → explain correctness → discuss indexes/transactions → cover edge cases and monitoring.

Dialect: Examples are primarily PostgreSQL. Oracle alternatives are called out; adapt date functions, upsert syntax, bind variables, and pagination syntax to the engine/version.

Question bank

SQL analytics: 5 scenarios

Banking transactions & consistency: 5 scenarios

Performance & production troubleshooting: 6 scenarios

Data modeling, migration & advanced SQL: 6 scenarios

SQL analytics

1. Second-highest salary per department

Solution: Use DENSE_RANK to include ties at the same salary rank.

SQL:

```
SELECT department_id, employee_id, salary
FROM (
  SELECT e.*,
         DENSE_RANK() OVER (
           PARTITION BY department_id ORDER BY salary DESC
         ) rn
  FROM employees e
) x
WHERE rn = 2;
```

Follow-up: DENSE_RANK has no gaps; RANK can skip ranks after ties.

2. Latest transaction per account

Solution: ROW_NUMBER selects the latest row for each account. Add a unique tie-breaker for deterministic results.

SQL:

```
SELECT account_id, transaction_id, txn_time, amount
FROM (
  SELECT t.*,
         ROW_NUMBER() OVER (
           PARTITION BY account_id
           ORDER BY txn_time DESC, transaction_id DESC
         ) rn
  FROM transactions t
) x
WHERE rn = 1;
```

Follow-up: Use RANK if all rows tied at the latest timestamp must be returned.

3. Customers with no orders in last 90 days

Solution: NOT EXISTS is a clear anti-join and avoids NULL surprises from NOT IN.

SQL:

```
SELECT c.customer_id, c.customer_name
FROM customers c
WHERE NOT EXISTS (
  SELECT 1 FROM orders o
  WHERE o.customer_id = c.customer_id
        AND o.order_date >= CURRENT_DATE - INTERVAL '90 days'
);
```

Follow-up: Oracle date equivalent: TRUNC(SYSDATE) - 90. Clarify whether 'no orders' means ever or within 90 days.

4. Running balance

Solution: A window SUM calculates the cumulative ledger total in transaction order.

SQL:

```
SELECT account_id, txn_id, txn_time, amount,
       SUM(amount) OVER (
         PARTITION BY account_id
         ORDER BY txn_time, txn_id
         ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
       ) running_balance
FROM ledger;
```

Follow-up: The ordering key must be stable; include an ID to break timestamp ties.

5. Duplicate emails

Solution: Normalize the email key consistently with the business rule, then group and filter duplicates.

SQL:

```
SELECT LOWER(TRIM(email)) email_key, COUNT(*) cnt
FROM customers
WHERE email IS NOT NULL
GROUP BY LOWER(TRIM(email))
HAVING COUNT(*) > 1;
```

Follow-up: After cleanup, enforce uniqueness with a unique index/constraint on the normalized key.

Banking transactions & consistency

6. Safe account-to-account transfer

Solution: Debit and credit must be atomic in one transaction. Lock rows consistently, validate amount/account state, and check the debit affected exactly one row.

SQL:

```
BEGIN;
SELECT account_id, balance
FROM accounts
WHERE account_id IN (:from_id, :to_id)
ORDER BY account_id
FOR UPDATE;

UPDATE accounts
SET balance = balance - :amount
WHERE account_id = :from_id
  AND balance >= :amount;
-- If row count != 1: ROLLBACK

UPDATE accounts
SET balance = balance + :amount
WHERE account_id = :to_id;
COMMIT;
```

Follow-up: Use rollback on any failure. Also validate amount > 0 and write an auditable ledger entry.

7. Prevent duplicate payment on retry

Solution: Persist a client idempotency key with a unique constraint. Duplicate requests return the previously stored result.

SQL:

```
CREATE TABLE payment_request (
  idempotency_key VARCHAR(100) PRIMARY KEY,
  request_hash VARCHAR(64) NOT NULL,
  payment_id BIGINT,
  status VARCHAR(20) NOT NULL
);

INSERT INTO payment_request
(idempotency_key, request_hash, status)
VALUES (:key, :hash, 'PROCESSING')
ON CONFLICT (idempotency_key) DO NOTHING;
```

Follow-up: PostgreSQL syntax. Oracle can use MERGE or handle duplicate-key exceptions. Reject same key with a different request hash.

8. Avoid lost updates

Solution: Use optimistic versioning or an atomic conditional update. A zero row count means the caller must reload/reconcile.

SQL:

```
UPDATE accounts
SET balance = balance - :amount,
    version = version + 1
WHERE account_id = :id
  AND balance >= :amount
  AND version = :expected_version;
```

Follow-up: Use SELECT FOR UPDATE when multiple validations/updates need a locked consistent view.

9. Find repeated same-payee same-amount transactions within 5 minutes

Solution: LAG compares each row with the previous row in the same account/payee/amount group. Treat matches as review signals, not proof of fraud.

SQL:

```
WITH x AS (
  SELECT t.*,
    LAG(txn_time) OVER (
      PARTITION BY account_id, payee_id, amount
      ORDER BY txn_time
```

```

        ) prev_time
FROM transactions t
)
SELECT *
FROM x
WHERE prev_time IS NOT NULL
      AND txn_time <= prev_time + INTERVAL '5 minutes';

```

Follow-up: Real rules also consider currency, status, channel, reversals, customer profile, and false positives.

10. Reconcile app totals with ledger

Solution: Aggregate each source by batch and FULL OUTER JOIN to find missing batches and amount mismatches.

SQL:

```

WITH a AS (
  SELECT batch_id, SUM(amount) total
  FROM app_payments GROUP BY batch_id
), l AS (
  SELECT batch_id, SUM(amount) total
  FROM ledger_entries GROUP BY batch_id
)
SELECT COALESCE(a.batch_id,l.batch_id) batch_id,
       a.total app_total, l.total ledger_total
FROM a FULL OUTER JOIN l USING (batch_id)
WHERE COALESCE(a.total,0) <> COALESCE(l.total,0);

```

Follow-up: Compare like-for-like statuses, currencies, cutoff timestamps, and reversal treatment.

Performance & production troubleshooting

11. Query became slow after data grew 10x

Solution: Capture the exact SQL and parameters, inspect actual plan, estimates, scans, joins, sorts, waits, and returned rows. Compare before/after and test with representative data.

SQL:

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT order_id, order_date, status
FROM orders
WHERE customer_id = :customer_id
      AND order_date >= :from_date
ORDER BY order_date DESC
LIMIT 50;
```

Follow-up: EXPLAIN ANALYZE executes the statement; be cautious with modifying SQL.

12. Index for customer order history

Solution: An equality filter followed by a date filter/order often benefits from a composite index with customer first.

SQL:

```
CREATE INDEX idx_orders_customer_date
ON orders (customer_id, order_date DESC);
```

Follow-up: Validate with the plan and workload; indexes consume storage and slow writes.

13. Slow deep pagination

Solution: OFFSET must skip many rows. Keyset pagination continues from the last seen sort key and uses a stable tie-breaker.

SQL:

```
SELECT order_id, order_date, status
FROM orders
WHERE customer_id = :customer_id
      AND (order_date, order_id) < (:last_date, :last_id)
ORDER BY order_date DESC, order_id DESC
LIMIT 50;
```

Follow-up: Index should align with customer_id and the ordering columns; handle nullable keys explicitly.

14. High DB CPU, normal traffic

Solution: Identify top SQL by total time and frequency, inspect plans and parameter skew, ORM SQL, statistics, polling, and recent releases.

SQL:

```
SELECT pid, state, wait_event_type, wait_event,
       now() - query_start AS duration, query
FROM pg_stat_activity
WHERE state <> 'idle'
ORDER BY duration DESC;
```

Follow-up: Oracle investigation commonly uses V\$SESSION and execution-plan/AWR tooling, depending on access.

15. Connection pool exhaustion

Solution: Check active sessions, long transactions, idle-in-transaction sessions, locks, and slow queries. Compare total connections across all app replicas with DB capacity.

SQL:

```
SELECT state, COUNT(*)
FROM pg_stat_activity
GROUP BY state;

SELECT pid, now() - xact_start AS xact_age, state, query
FROM pg_stat_activity
WHERE xact_start IS NOT NULL
ORDER BY xact_age DESC;
```

Follow-up: Increasing pool size blindly can overload the DB. Check app-side pool metrics and connection leaks.

16. Deadlock scenario

Solution: Deadlocks arise from cyclic lock waits. Make all code paths lock resources in a consistent order, shorten transactions, index update predicates, and use bounded retry for safe transactions.

SQL:

```
SELECT account_id
FROM accounts
WHERE account_id IN (:a, :b)
ORDER BY account_id
FOR UPDATE;
```

Follow-up: Retry the entire transaction, not just the failed statement, and use backoff/jitter.

Data modeling, migration & advanced SQL

17. Partition a large transaction table by date

Solution: Range partitioning can help retention and partition pruning when queries filter by date. It does not replace indexes or good query design.

SQL:

```
CREATE TABLE txn_archive (  
    txn_id BIGINT,  
    txn_date DATE NOT NULL,  
    amount NUMERIC(18,2)  
) PARTITION BY RANGE (txn_date);  
  
CREATE TABLE txn_2026_q3 PARTITION OF txn_archive  
FOR VALUES FROM ('2026-07-01') TO ('2026-10-01');
```

Follow-up: If queries omit the partition key, many partitions may still need scanning.

18. Oracle to PostgreSQL migration with minimal downtime

Solution: Map types and semantics, rehearse, bulk-copy baseline, capture ongoing changes if required, validate counts/checksums/aggregates, cut over with lag checks and rollback plan.

SQL:

```
-- Compare at agreed snapshot/cutoff  
SELECT COUNT(*) FROM source_table;  
SELECT COUNT(*) FROM target_table;  
  
SELECT status, COUNT(*), SUM(amount)  
FROM target_table  
GROUP BY status;
```

Follow-up: Check sequences, timestamp/time zone, numeric precision, case sensitivity, procedures, and isolation differences.

19. Delete millions of old audit rows safely

Solution: Use a retention policy and bounded batches; monitor locks, WAL/undo, replication lag and bloat. If date-partitioned, dropping/detaching old partitions may be more efficient.

SQL:

```
WITH batch AS (  
    SELECT id FROM audit_log  
    WHERE created_at < :cutoff  
    ORDER BY id  
    LIMIT 5000  
)  
DELETE FROM audit_log a  
USING batch b  
WHERE a.id = b.id;
```

Follow-up: Run repeatedly with pacing; verify backups, legal retention, and dependent consumers first.

20. Upsert profile by external ID

Solution: Enforce uniqueness on the business key and use an upsert. Decide whether incoming NULL values should overwrite existing data.

SQL:

```
INSERT INTO customer_profile (external_id, full_name, updated_at)  
VALUES (:external_id, :full_name, CURRENT_TIMESTAMP)  
ON CONFLICT (external_id)  
DO UPDATE SET  
    full_name = EXCLUDED.full_name,  
    updated_at = CURRENT_TIMESTAMP;
```

Follow-up: PostgreSQL syntax; Oracle supports MERGE.

21. Why NOT IN can fail with NULL

Solution: SQL uses three-valued logic. A NULL in the subquery can make NOT IN evaluate UNKNOWN. NOT EXISTS is usually safer for anti-joins.

SQL:

```
SELECT c.customer_id
FROM customers c
WHERE NOT EXISTS (
  SELECT 1 FROM blocked_customers b
  WHERE b.customer_id = c.customer_id
);
```

Follow-up: If using NOT IN, filter NULLs in the subquery and verify nullable outer keys.

22. Detect overlapping bookings

Solution: Self-join rows for the same resource and test interval intersection. Decide if touching endpoints count as overlap.

SQL:

```
SELECT a.booking_id, b.booking_id, a.resource_id
FROM bookings a
JOIN bookings b
  ON a.resource_id = b.resource_id
 AND a.booking_id < b.booking_id
 AND a.start_time < b.end_time
 AND b.start_time < a.end_time;
```

Follow-up: For PostgreSQL, range types and exclusion constraints can help prevent overlaps.

Senior interview quick revision

- Clarify NULL handling, duplicate/tie behavior, time zones, date boundaries, and deterministic ordering.
- Use execution plans before recommending indexes. Measure with representative data and workload.
- For money movement, emphasize atomic transactions, conditional updates, idempotency, constraints, and audit trails.
- For production incidents, correlate API latency with slow SQL, wait events, locks, pool metrics, CPU/I/O, and recent deployments.
- Use bounded batches for large updates/deletes; plan for locks, WAL/undo, replication lag, and rollback.
- Validate migration results using counts, checksums, key aggregates, and application-level reconciliation.